

PONTIFÍCIA UNIVERSIDADE CATÓLICA

João Victor Russo Marquito

IMPLEMENTAÇÃO DE SOCKETS TCP E UDP EM PYTHON

Belo Horizonte
2026

1. Introdução

Este relatório apresenta a implementação prática de três modelos fundamentais de programação em sockets usando a pilha TCP/IP: UDP (sem conexão), TCP Simples (sequencial) e TCP Concorrente (multithread). O objetivo principal é demonstrar, através de códigos funcionais e testes de execução local, o comportamento, as garantias de entrega e as diferenças cruciais na arquitetura de comunicação em rede na camada de transporte. Cada modelo é explorado detalhando seus mecanismos internos de conexão e fluxo de dados.

2. Diferença entre UDP e TCP

A camada de transporte da Internet disponibiliza dois protocolos principais para a comunicação entre processos: o UDP e o TCP. A escolha entre eles define como os dados trafegam e como a aplicação lida com a rede.

2.1 UDP - User Datagram Protocol

O UDP é um protocolo de transporte leve e "de melhor esforço". Ele não estabelece uma sessão formal antes de enviar os dados (não orientado à conexão). Sua principal característica é a baixa latência, uma vez que elimina o tráfego adicional gerado por verificações rigorosas de erro, ordenação de pacotes ou solicitações de retransmissão.

Aplicações: É o formato ideal para cenários onde a velocidade é mais crítica que a precisão absoluta. Em partidas competitivas de jogos online, pacotes UDP são disparados constantemente para atualizar a posição dos jogadores no servidor. Se um pacote (tick) for perdido na rede, é preferível simplesmente aceitar a perda e processar a próxima coordenada imediatamente, mantendo o ping baixo e a fluidez, em vez de pausar o jogo esperando a retransmissão de um dado que já ficou obsoleto.

2.2 TCP - Transmission Control Protocol

O TCP é um protocolo altamente confiável e orientado à conexão. Antes de qualquer troca de dados, ele utiliza um processo chamado *three-way handshake* para estabelecer um canal seguro entre cliente e servidor. O protocolo garante que os pacotes cheguem exatamente na ordem em que foram enviados e sem perdas estruturais, gerenciando internamente o reenvio de qualquer informação que tenha falhado no trajeto.

Aplicações: No desenvolvimento do backend de um sistema web corporativo utilizando Spring Boot, por exemplo, as requisições REST da API para persistir dados em um banco MySQL dependem unicamente do TCP. Isso garante que a carga de dados (como o cadastro de um produto ou um pedido) chegue completa e sem corrupção.

2.3 Comparação UDP x TCP

Característica	UDP	TCP
Conexão	Não orientado	Orientado à conexão
Confiabilidade	Não confiável	Confiável
Velocidade	Mais rápido	Mais lento

3. UDP

3.1 Código do Servidor (UDPServer.py)

```
from socket import *

serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind('', serverPort)
print('The server is ready to receive')

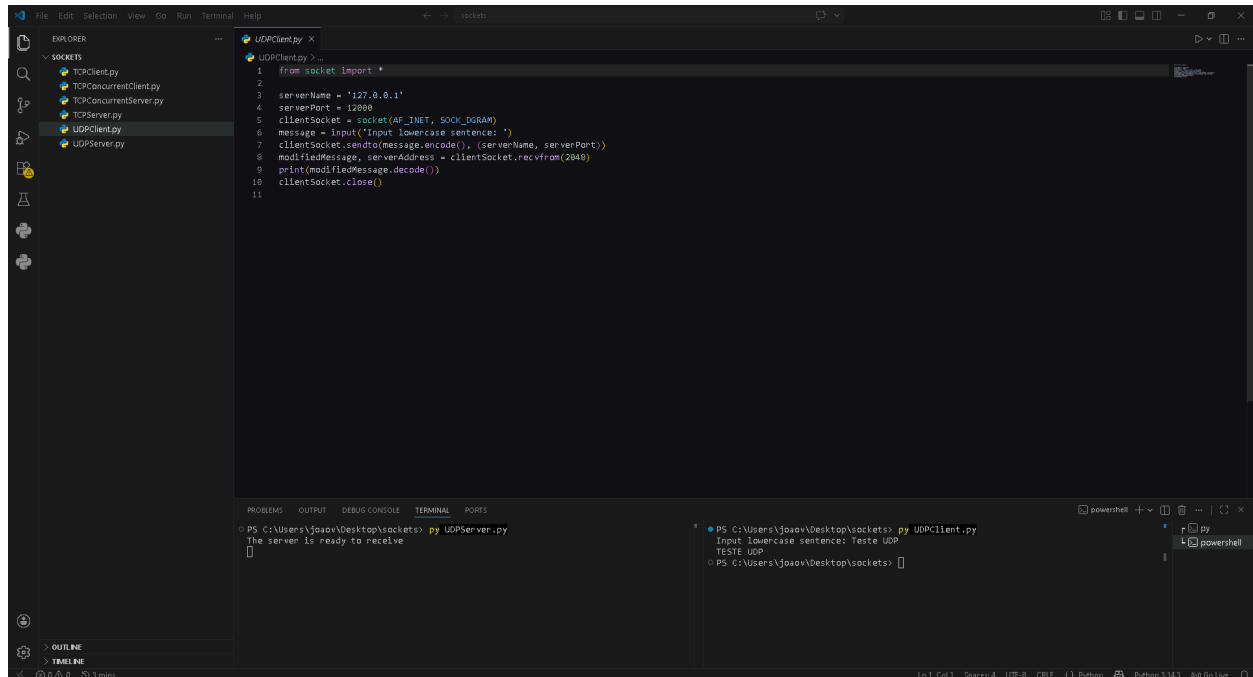
while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(), clientAddress)
```

3.2 Código do Cliente (UDPClient.py)

```
from socket import *

serverName = '127.0.0.1'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_DGRAM)
message = input('Input lowercase sentence: ')
clientSocket.sendto(message.encode(), (serverName, serverPort))
modifiedMessage, serverAddress = clientSocket.recvfrom(2048)
print(modifiedMessage.decode())
clientSocket.close()
```

3.3 Execução do Terminal



3.4 Explicação do Funcionamento

Neste modelo, o código demonstra uma troca simples de mensagens baseada em datagramas sem nenhum vínculo contínuo. O servidor escuta a porta 12000 usando o método `recvfrom()`, que captura passivamente qualquer pacote que chegue, extraíndo a mensagem e o endereço (IP e porta) de origem. Do outro lado, o cliente usa `sendto()` para mandar a string diretamente para o endereço 127.0.0.1. Após o servidor processar o texto (convertendo para maiúsculas) e devolver a resposta com base no endereço capturado, os sockets são liberados. Não há

encerramento de conexão, pois ela nunca existiu formalmente.

4. TCP - Cliente e Servidor Simples

4.1 Código do Servidor (TCPServer.py)

```
from socket import *

serverPort = 12000
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
print('The server is ready to receive')

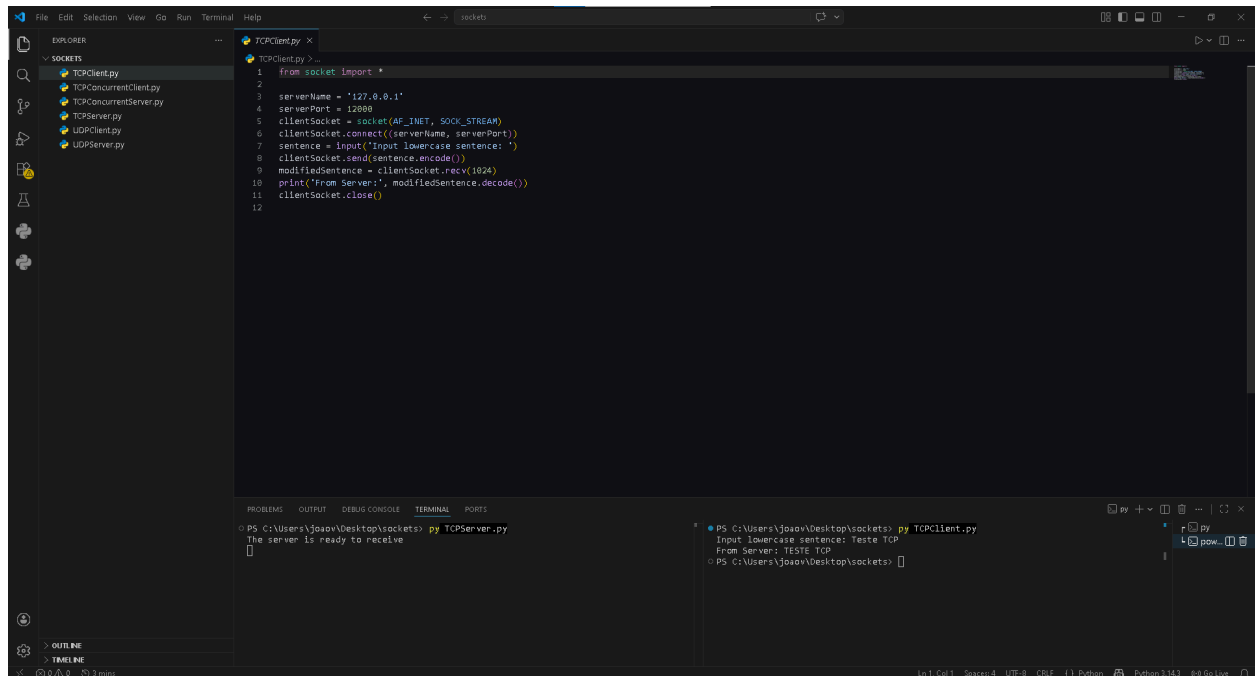
while True:
    connectionSocket, addr = serverSocket.accept()
    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.encode())
    connectionSocket.close()
```

4.2 Código do Cliente (TCPClient.py)

```
from socket import *

serverName = '127.0.0.1'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, serverPort))
sentence = input('Input lowercase sentence: ')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print('From Server:', modifiedSentence.decode())
clientSocket.close()
```

4.3 Execução



4.4 Explicação do Funcionamento

Ao contrário do UDP, a dinâmica aqui exige uma sincronia prévia. O servidor TCP usa a instrução `listen(1)` para entrar em modo de escuta passiva. Quando o cliente invoca o método `connect()`, o handshake ocorre e um "tubo" lógico de dados é estabelecido. A grande diferença estrutural é que a função `accept()` do servidor gera um novo socket dedicado (`connectionSocket`) exclusivamente para interagir com esse cliente específico. Após a troca da mensagem em letras minúsculas pela maiúscula, o servidor invoca `connectionSocket.close()`, derrubando imediatamente a sessão e liberando o fluxo principal para aceitar um eventual próximo cliente.

5. TCP - Servidor Concorrente

5.1 Código do Servidor (TCPConcurrentServer.py)

```
import socket
import _thread

HOST = '127.0.0.1'
PORT = 50000

def conectado(con, cliente):
    print('\nCliente conectado:', cliente)
    while True:
        msg = con.recv(1024)
```

```

        if not msg:
            break
        print('\nCliente...:', cliente)
        print('Mensagem..:', msg.decode())
    print('\nFinalizando conexao do cliente', cliente)
    con.close()
    _thread.exit()

tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
orig = (HOST, PORT)
tcp.bind(orig)
tcp.listen(1)
print('\nServidor TCP concorrente iniciado no IP', HOST, 'na porta',
PORT)

while True:
    con, cliente = tcp.accept()
    print('\nNova thread iniciada para essa conexao')
    _thread.start_new_thread(conectado, tuple([con, cliente]))

```

5.2 Código do Cliente (TCPConcurrentClient.py)

```

import socket

HOST = '127.0.0.1'
PORT = 50000

tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
destino = (HOST, PORT)
tcp.connect(destino)

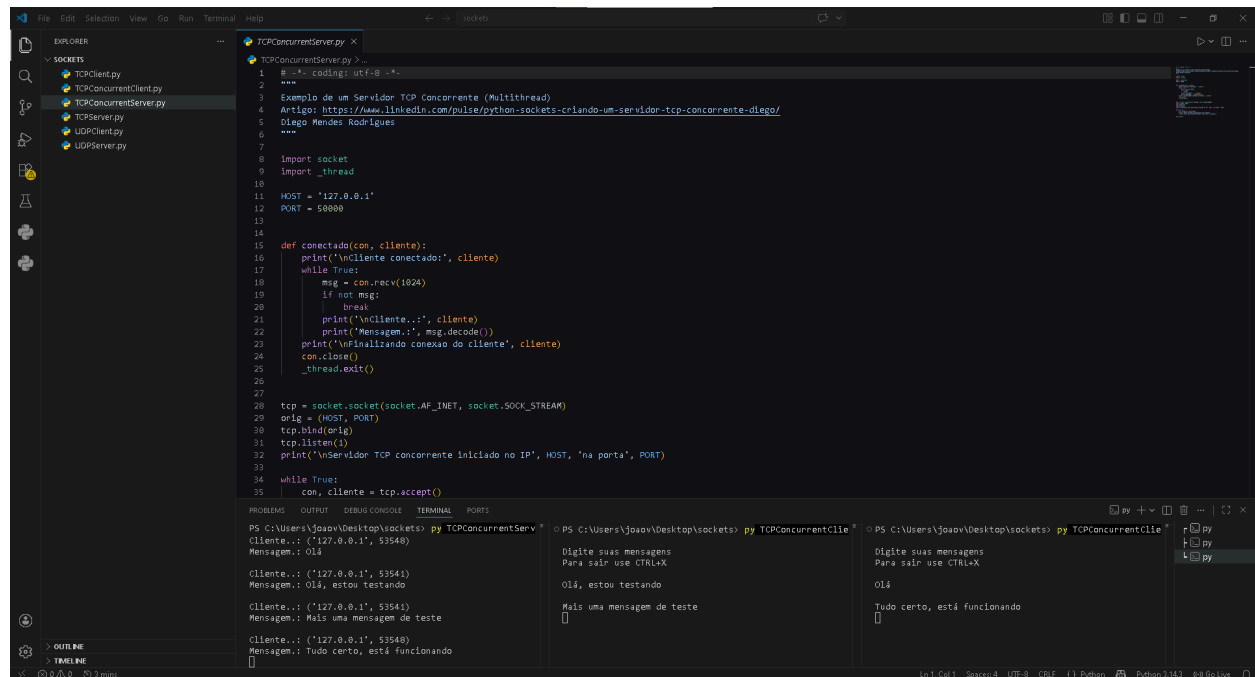
print('\nDigite suas mensagens')
print('Para sair use CTRL+X\n')

try:
    mensagem = input()
    while mensagem != '\x18':
        tcp.send(str(mensagem).encode())
        mensagem = input()
except EOFError:
    pass

tcp.close()

```

5.3 Execução



```
1 # -*- coding: utf-8 -*-
2 """
3 Exemplo de um Servidor TCP Concorrente (Multithread)
4 Artigo: https://www.linkedin.com/pulse/python-sockets-criando-um-servidor-tcp-concorrente-diego/
5 Diego Mendes Rodrigues
6 """
7
8 import socket
9 import _thread
10
11 HOST = '127.0.0.1'
12 PORT = 50000
13
14
15 def conectado(con, cliente):
16     print('\nCliente conectado:', cliente)
17     while True:
18         msg = con.recv(1024)
19         if not msg:
20             break
21         print('\nCliente...:', cliente)
22         print('Mensagem: ', msg.decode())
23         print('\nFinalizando conexao do cliente', cliente)
24         con.close()
25         _thread.exit()
26
27
28 tcp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
29 orig = (HOST, PORT)
30 tcp.bind(orig)
31 tcp.listen(1)
32 print('\nServidor TCP concorrente iniciado no IP', HOST, 'na porta', PORT)
33
34 while True:
35     con, cliente = tcp.accept()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Joao\Desktop\sockets> py TCPConcurrentServ.py
Cliente.: ('127.0.0.1', 53548)
Mensagem.: Olá
Cliente.: ('127.0.0.1', 53541)
Mensagem.: Olá, estou testando
Cliente.: ('127.0.0.1', 53541)
Mensagem.: Mais uma mensagem de teste
Cliente.: ('127.0.0.1', 53548)
Mensagem.: Tudo certo, está funcionando

PS C:\Users\Joao\Desktop\sockets> py TCPConcurrentClien.py
Digite suas mensagens
Para sair use CTRL+X
Olá, estou testando
Mais uma mensagem de teste
Tudo certo, está funcionando

5.4 Explicação do Funcionamento

O modelo sequencial anterior apresenta um gargalo estrutural: ele bloqueia a aplicação, atendendo apenas um usuário por vez até que a conexão seja fechada. Este servidor TCP Concorrente resolve o problema delegando o processamento. O loop principal do servidor (while True) fica focado exclusivamente em aceitar novas conexões com `accept()`. Assim que um cliente se conecta na porta 50000, o servidor invoca `_thread.start_new_thread()`, repassando o socket daquela conexão recém-criada para uma função isolada (`conectado()`). Isso permite que os múltiplos clientes nos terminais se mantenham conectados de forma persistente. Cada um possui sua própria thread rodando em segundo plano, trocando mensagens de maneira independente e simultânea, sem que um bloqueie o fluxo de rede do outro.

6. Conclusão

A implementação prática destes três modelos de sockets evidenciou de forma clara as particularidades e os compromissos (trade-offs) da camada de transporte na arquitetura TCP/IP. Ficou demonstrado que não existe um protocolo universalmente superior, mas sim escolhas arquiteturais baseadas nos requisitos específicos de cada sistema.

O UDP validou sua utilidade em cenários de disparo rápido e sem estado. Sua ausência de overhead de conexão o torna o modelo ideal para aplicações onde a baixa latência é prioritária

e a perda ocasional de pacotes é tolerável, como na constante atualização de posições e ações em servidores de jogos online competitivos (como o Counter-Strike 2).

Por outro lado, o TCP demonstrou seu valor através da estabilidade e confiabilidade. O handshake inicial e a garantia de entrega ordenada de pacotes o tornam indispensável para o tráfego de informações sensíveis. É a base fundamental para o desenvolvimento de sistemas robustos, como na construção de APIs RESTful estruturadas em Spring Boot que necessitam persistir e consultar dados de forma segura em bancos relacionais como o MySQL.

Por fim, a evolução do TCP Simples para o TCP Concorrente revelou a importância do paralelismo para a escalabilidade de rede. O uso de threads provou que é perfeitamente viável manter a confiabilidade inegociável do TCP enquanto o servidor atende a múltiplos clientes simultaneamente, eliminando o gargalo de bloqueio síncrono. O domínio da aplicação prática desses conceitos é um pilar essencial para o projeto de software distribuído e para a otimização de infraestruturas de rede no mundo real.

7. Referências

Slides REDES3_ES_TCP_Sockets_Parte2.pdf. Disciplina de Redes de Computadores.